Module 3, Lecture 1 — Production-Level System Architecture and Technical Design

> **Course:** Staff Engineer / Technical Lead Mentorship **Module:** 3 — System architecture, technical design, API contracts, ERDs, and data flows **Lecture:** 1 of Module 3 **Case study:** "Spendwise," the personal finance app from Module 2 (React Native + FastAPI + PostgreSQL) **Format:** A one-way lecture. Everything is demonstrated for you, including the exercise, a review of a typical attempt, and a reference solution. Nothing here waits for your reply. **Scope note:** This lecture covers the *architecture and the technical design document*. The detailed ERD and the full API contract are the subject of later lectures in this module. I will sketch them here only where the architecture depends on them.

Part 1 — Learning objective

After this lecture you should be able to:

1. **Extract the architecturally significant requirements** from a PRD, the few requirements that actually shape the structure of the system.
2. **Ask the questions** a lead asks *before* choosing any technology.
3. **Compare architecture alternatives** against requirements and constraints, and choose one with a defensible reason and a condition for revisiting it.
4. **Draw component boundaries** and say what each component owns and what it must never do.
5. **Trace data flows** end to end and identify what can fail at every step.
6. **Think through security, scalability, and operations** as design inputs rather than afterthoughts.
7. **Turn the reasoning into a Technical Design Document (TDD)** and **Architecture Decision Records (ADRs)** of the right size.

Part 2 — Real-world scenario

The Spendwise PRD (v0.1) has been reviewed and approved by Meera. You are the lead, and in this freelance project you are also the whole engineering team. The next morning you open a blank repository.

Here is what you have from the PRD, in short:

- **Outcome:** pilot users record expenses on the day they happen and see where their money went.

- **Must requirements:** accounts, record/edit/delete expense, fixed categories, monthly summary by category, account and data deletion, usage events.
- **Selected NFRs:** save responds in 500 ms (p95) at 50 concurrent users; 99.5% monthly availability; totals always equal the sum of undeleted expenses; exactly one expense stored on a double tap or a retry; RPO 24 h and RTO 8 h; passwords never readable; each user sees only their own data; account deletion removes data within 30 days and backups expire within 35 days.
- **Exclusions:** no bank import, no offline-first, no shared households, no multi-currency conversion, no custom categories.
- **Assumptions:** about 10 pilot users, INR only, users mostly online when they log.
- **Constraints you know:** one engineer (you), a couple of months, iOS and Android, a small budget, and a stack you already chose to learn: React Native, FastAPI, PostgreSQL.

Now think about what most developers do next. They open a tutorial for "FastAPI + React Native + Docker + Redis + JWT", copy a project structure, and start creating folders named `controllers`, `services`, `models`. They are not wrong to want to move. But they have skipped the most valuable hour of the project: **deciding what has to be true about the structure, and why.**

That hour is today's lecture.

Part 3 — The lead's thought process

3.1 What "architecture" actually means

People use "architecture" for everything from boxes-and-arrows diagrams to folder structure. I use this working definition:

> **Architecture is the set of structural decisions that are expensive to change later**, and the reasons behind them.

That definition does useful work. It tells you *which* decisions deserve the attention: the ones that are hard to reverse. It is the same "one-way door" idea from Lecture 1, now applied to system structure.

It helps to separate three levels:

Level	Question it answers	Example in Spendwise	Cost to change
Architecture	What are the major parts, what does each own, how do they talk, where does data live?	One API service, one PostgreSQL database, a mobile client that never touches the database	High
Design	How is one part structured internally?	Inside the API: routers, services, repositories; how the save-expense transaction works	Medium
Implementation	The code itself	Function names, library choices for date parsing	Low

A lead spends the *most* thinking time at the top and the *least* at the bottom. A common beginner pattern is the reverse: long debates about folder structure (cheap to change) and no thought about whether the client may talk to the database directly (expensive to change).

Architecturally Significant Requirements (ASRs). Not every requirement shapes the architecture. "A note can be at most 200 characters" does not. "Exactly one expense stored on retry" does, because it affects the API contract and the database design. "99.5% availability" does, because it affects hosting and deployment. An ASR is a requirement (usually a non-functional one) or a constraint that **would force a different structure if it changed.** The first job of design is to find the ASRs. Everything else follows from them.

What an inexperienced developer misses: they start from technology and ask "what can I build with this stack?" instead of "what must be true about this system?"

What an experienced lead investigates: which two or three requirements are the real drivers, and what would change in the architecture if each driver changed.

DBA connection. You already do this in capacity planning. You don't choose a storage layout first and then discover the workload. You ask about read/write ratio, growth, recovery requirements, and *then* design. Architecture is the same thinking, applied across the whole stack.

3.2 The questions a lead asks before selecting architecture

I use eight groups of questions. I don't ask them in a meeting as a script. I use them as a checklist for myself and for conversations with the client.

Group	Questions	What it protects you from
1. Drivers (ASRs)	Which requirements are hard to meet and would change the structure? What are the quality-attribute priorities when they conflict (for example consistency over speed)?	Designing for the wrong goal
2. Load and size	How many users, requests per second, rows, and bytes, today and at a realistic upper bound? How "bursty" is the traffic?	Over-building or under-building for scale
3. Data	Who owns each piece of data? How sensitive is it? What consistency does it need? How long is it kept? How is it deleted?	Data corruption, privacy failures, painful migrations
4. Change	What is *certain* to change soon? What is certain *not* to change? Which parts are experiments?	Rigid core, or flexibility in the wrong places
5. Failure	What can fail? What is the blast radius? What does the user see? What is the worst thing that could happen to the data?	Surprise outages and data loss
6. Operations	Who deploys, monitors, and gets woken at 3 a.m.? How do we back up, restore, roll back, and observe?	Beautiful designs that nobody can run
7. Team and cost	How many people, with what skills? What is the monthly budget? What happens if the lead is unavailable?	Choosing something the team cannot sustain
8. Compliance and constraints	What laws, store rules, data-residency or contractual rules apply?	Late legal or store-review surprises

Doing the load estimate: a back-of-envelope calculation. This is one of the most valuable ten-minute habits. Let's do it for Spendwise, with a generous upper bound of 1,000 users:

```
Writes:   1,000 users × 10 expenses/day            = 10,000 expenses/day
          10,000 / 86,400 seconds                   ≈ 0.12 writes/second average
          Assume evening peak = 20× average         ≈ 2.5 writes/second at peak
Reads:    Assume 5 summary/list reads per write     ≈ 12 reads/second at peak
Rows:     10,000/day × 365                          ≈ 3.65 million expense rows
per year
Storage:  ~100–150 bytes per row plus indexes,
          call it ~300 bytes all-in                 ≈ 1 GB per year (order of
magnitude)
```

Look at what this tells us. **Even at 1,000 users, one modest API instance and one PostgreSQL instance have enormous headroom.** The load is not the hard part of this project. The hard parts are:

1. **Correctness** (money, duplicates, deletes, month boundaries).
2. **Security and privacy** (financial data, account deletion).
3. **Operability** (a solo engineer must run it reliably).
4. **Mobile release constraints** (you cannot instantly update apps already installed on phones).

That is exactly the kind of insight a back-of-envelope calculation gives you. It stops you from spending your effort on Kubernetes and caching when the real risks are a duplicate-save bug and an untested backup.

(Those numbers are rough on purpose. The point is the order of magnitude, not precision. When you can't estimate, you say so and list it as an assumption.)

3.3 Alternatives considered

A lead never presents a single option. If you cannot name at least two or three realistic alternatives, you probably have not understood the problem space. Here are the real candidates for Spendwise.

Option A: Modular monolith. One React Native app talks over HTTPS to **one** FastAPI service, which talks to **one** PostgreSQL database. The FastAPI codebase is split into internal modules (auth, expenses, summaries, and so on) with clear boundaries, but it is deployed as a single unit.

Option B: Microservices. Separate services for authentication, expenses, reporting, and notifications, each with its own deployment, and perhaps its own database, communicating over HTTP or a message broker.

Option C: Serverless functions plus a managed database. Each endpoint is a function (for example AWS Lambda behind API Gateway), with PostgreSQL as a managed database.

Option D: Backend-as-a-Service (BaaS). Use a platform such as Supabase or Firebase. The client talks to the platform directly. Business rules are expressed in database policies and small server functions. *(Supabase is built on PostgreSQL and uses Row-Level Security policies for access control. Firebase uses a different, document-oriented data model.)*

Option E: Local-first. The phone has its own database (for example SQLite), works fully offline, and syncs with a server in the background.

Now I evaluate them against the drivers. The evaluation matters more than the verdict, so I'll show it in a table, and you can disagree with my scoring as long as you can state your own reasoning.

Driver or constraint	A. Modular monolith	B. Microservices	C. Serverless + DB	D. BaaS	E. Local-first
Exactly-once expense on retry (NFR-5)	Easy: one transaction, unique constraint	Harder: cross-service consistency	Easy, but retries by the platform can double-invoke functions	Possible via DB constraints	Hard: needs conflict resolution
Totals always equal sum (NFR-4)	Easy: one database, SQL aggregates	Harder if data is split	Easy	Easy (SQL)	Hard: two copies of truth
Solo engineer must operate it	**Good:** one thing to deploy and watch	**Poor:** many things, many failure modes	Good, but debugging distributed functions is harder	**Best:** platform operates much of it	Poor: client and server complexity
Business rules in one testable place	**Good**	Spread across services	Spread across functions	Rules live in policies and functions, harder to test and review	Spread across client and server
Account deletion and data export control (NFR-8)	Full control	Must coordinate across services	Full control	Possible, platform-dependent	More places hold data
Offline use	Not supported (excluded in v1)	Not supported	Not supported	Limited	**Best** (but excluded)
Lock-in and exit cost	Low: standard Postgres, standard code	Low to medium	Medium: provider-specific glue	**Higher:** logic and auth tied to platform	Medium
Learning goal (FastAPI, SQL, architecture)	**Strong**	Distracting	Moderate	Weak	Moderate

Driver or constraint	A. Modular monolith	B. Microservices	C. Serverless + DB	D. BaaS	E. Local-first
Fit for growth to ~1,000 users	Ample	Overkill	Ample	Ample	Ample

My decision: Option A, the modular monolith. Notice the reasons:

1. Drivers NFR-4 and NFR-5 are **data correctness** requirements, and correctness is easiest when there is *one database and one place where the rules are enforced.*
2. The team is one person. Every additional deployable unit is an extra thing that can break at night.
3. The load estimate shows no need for horizontal complexity.
4. Business rules (alerts, deletion, totals) live in ordinary, testable Python code that a future developer can read.

An honest note about Option D. For a *solo freelancer on a tight timeline*, a BaaS is a perfectly respectable choice, and for some clients it is the better one. It removes a lot of operational work. I did not choose it here because (a) the business rules around money and deletion benefit from being in reviewable code, (b) this project doubles as your learning of FastAPI and architecture, and (c) the exit cost is lower with a standard stack. **If the client's priority were "launch in three weeks and don't care about owning the stack," I would choose differently.** That is exactly what "context-dependent" means.

And a caution from Lecture 1. If your reason for choosing a stack is partly that *you want to learn it*, say so openly to the client and weigh the risk. Here the client's needs and your learning goals happen to point the same way, and I have still written the client-facing reasons first.

When would this decision change? (The "revisit if" list.)

- If more than one team must work on this independently with separate release schedules, consider splitting services.
- If a feature appears that needs very different scaling or runtime characteristics (for example heavy receipt-image processing), extract *that one part* as a separate worker or service.
- If offline use becomes a Must, revisit local-first for the mobile client.
- If operating the backend becomes the bottleneck for a small team, revisit BaaS or more managed services.

Why not microservices "for the future"? Microservices solve *organisational* scaling problems (many teams) and *independent scaling* problems. You have neither. What you would get

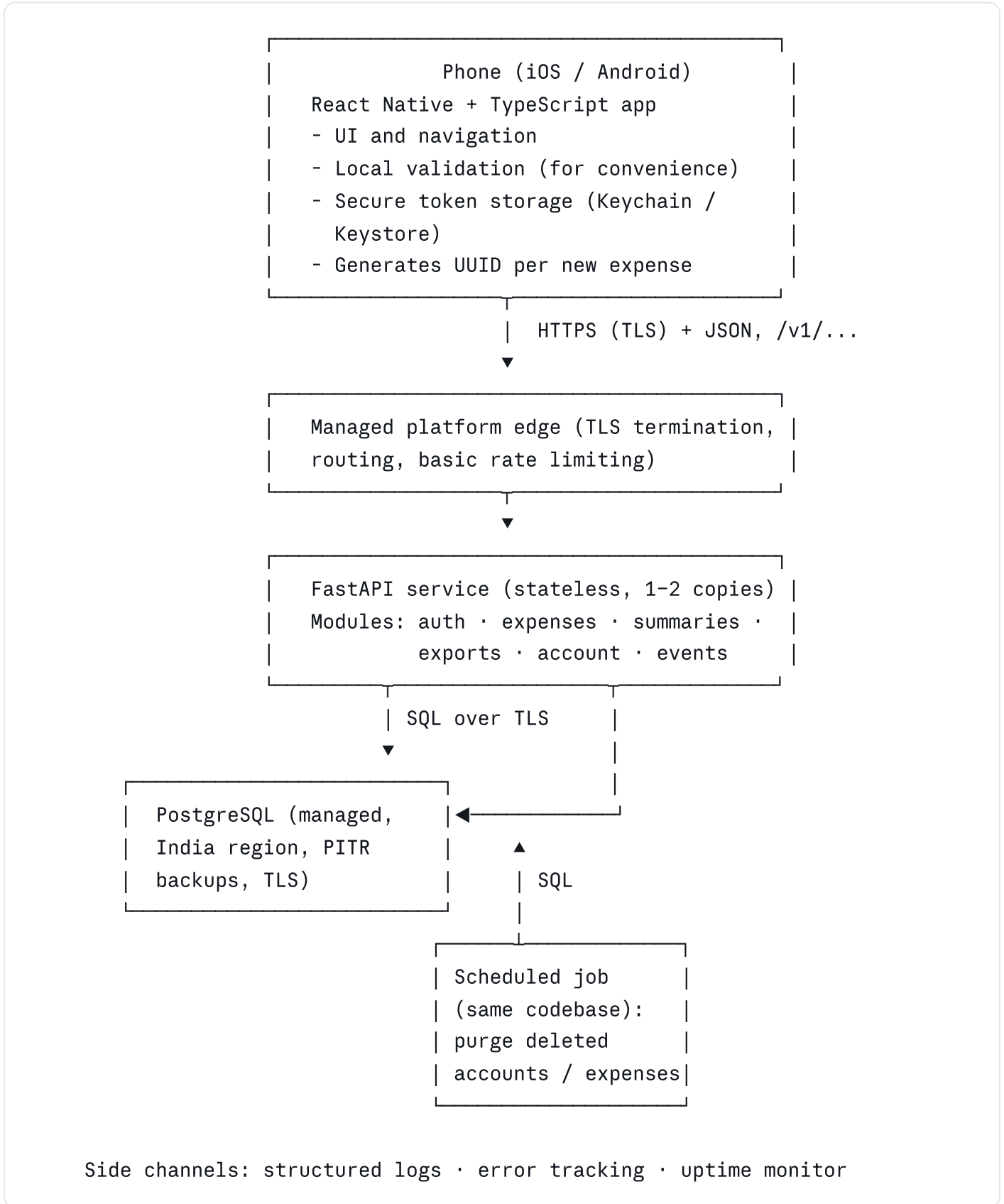
instead is network calls where function calls used to be, distributed failure modes, distributed transactions, and many deployments, which is the cost without the benefit. A well-known failure pattern is the **distributed monolith**: a system split into services that still have to be deployed and changed together, so it has the cost of both worlds.

Why *modular* monolith, not just "a monolith"? A monolith without internal boundaries becomes a ball of mud. A modular monolith has the *deployment* simplicity of one unit and the *logical* separation of modules with explicit interfaces. If you ever need to extract a service, the seam is already there.

3.4 Component boundaries

A **boundary** is a line where one part of the system stops and another starts, with a defined contract between them. Good boundaries make three things true: each part has a **single reason to change,** each piece of data has a **single owner,** and the **contract** across the boundary is explicit.

Here is the Spendwise architecture, in a diagram:



Let me explain the **rules of this architecture,** because the rules are the design, not the boxes.

Rule 1: The client never talks to the database. Only the API touches PostgreSQL. This is a boundary that is *very* hard to add later. Once a mobile app is in people's pockets with direct

database access, you cannot take it back. The API is the single place where authentication, authorisation, validation, and business rules are enforced.

Rule 2: The client is untrusted. Anything the app checks (amount format, required fields) is for the user's convenience. The server re-checks everything. Anyone can call your API with a script and bypass the app entirely. Beginners often put the "real" validation in the app and forget that the API is public.

Rule 3: The API is stateless. It holds no user session in memory and no files on local disk that matter. All durable state lives in PostgreSQL. Why? Because then you can run two copies, restart one, or replace it during a deploy without losing anything. Statelessness is the cheapest insurance for both availability and scaling.

Rule 4: One database owner. The API's modules share one schema, but each table has one *owning module* that is the only code allowed to write to it. For example, only the `expenses` module writes the `expenses` table. Other modules call its functions. This is an internal boundary that mirrors what a microservice boundary would be, without the network.

Rule 5: Business rules live in the server's service layer, not in route handlers and not in SQL scattered everywhere. Within each FastAPI module I use a simple layering:

```
Router (HTTP concerns: parse request, status codes)
   | calls
   ▼
Service (business rules: "an expense must have a valid category", the
         save-expense transaction, alert logic later)
   | calls
   ▼
Repository (SQL: queries and inserts; always filters by the current user)
   |
   ▼
PostgreSQL
```

Dependencies point **downwards only.** A repository never calls a router. This isn't ceremony. It means the business rules can be tested without HTTP, and the SQL can be reviewed in one place. **A caution:** don't build abstractions for their own sake. For a project this size, three layers per module is plenty. Do not add generic "base repository" frameworks, dependency-injection containers, or interfaces for things that have only one implementation.

Rule 6: The contract between client and server is the OpenAPI document. FastAPI generates an OpenAPI specification automatically from your request and response models (written with Pydantic). You can generate TypeScript types for the React Native app from it, so

the client and server cannot silently drift apart. That turns the boundary into something checked by tools, not by memory.

Rule 7: Background work is a separate entry point, not a separate service. The purge job runs as a scheduled process using the same codebase and database access layer, but it is not part of the request path. If it fails, no user request is affected. (Avoid running scheduled tasks *inside* each API process: with two API copies, the job would run twice.)

What lives on the client side? Inside the React Native app I also keep boundaries: screens do not call the network directly; they use a small API client module. A server-state library such as TanStack Query handles caching, loading states, and retries. Tokens go in the platform's secure storage (iOS Keychain, Android Keystore), **never** in plain local storage. (The React Native documentation currently recommends starting new apps with a framework such as Expo; whether to use Expo or a bare React Native project is an ADR candidate. I'd verify the current recommendation when making that decision.)

3.5 Data flows

A data flow is the path a piece of information takes through the system. You trace the flows for the **important** operations and ask at each step: *what is the data here, who owns it, what can fail, and how do we recover?*

Flow 1: Save an expense (the most important flow in the product)

```
 1. User taps Save in the app
 2. App validates the form (convenience only)
 3. App creates a new UUID for this expense ("client-generated ID")
    and sends:  POST /v1/expenses
                Authorization: Bearer <access token>
                { id, amount_minor, category_id, spent_on, note }
 4. Edge/platform receives it over TLS and forwards it
 5. API authenticates the token, identifies the user
 6. API validates the body with Pydantic (types, ranges, lengths)
 7. Service starts a database transaction
 8. Repository runs: INSERT ... ON CONFLICT (id) DO NOTHING
        - New row inserted     → continue
        - Row already existed  → this is a retry; read it back
             - same user, same content → return it (idempotent replay)
             - another user, or different content → return an error
 9. Transaction commits
10. API returns 201 (created) or 200 (replay) with the expense
11. App updates its screen and its cached monthly totals
```

Let me draw out the five design ideas hidden in these eleven steps.

(a) The ID is created by the client. This is what makes "exactly one expense on retry" possible. If the ID were created by the server, a retry would look like a brand-new request and you could not tell. With a client-generated ID, the retry carries the *same* ID, and the database's primary key rejects the duplicate.

(b) The database is the last line of defence. The unique constraint on `id` guarantees no duplicate row *even if* two requests arrive at the same instant on two API copies and even if the application code has a bug. **Constraints enforced by the database do not depend on the discipline of every developer.** This is your DBA instinct, and it is the correct one.

(c) Why `ON CONFLICT DO NOTHING` and then a read-back? Think about the nastiest failure: the server saves the expense and the transaction commits, but the response is lost because the phone's connection drops. The app thinks it failed and retries. Without idempotency you get a duplicate. With it, the retry finds the existing row and returns it. The user sees one expense, and nobody lost anything.

(d) What about a retry with the same ID but *different* content? That means a bug or tampering. Return an error, don't silently overwrite. Similarly, if the ID belongs to *another user*, return a generic error without revealing that the ID exists elsewhere.

(e) The client chooses `spent_on` as a calendar date. More on this in the data decision below. It matters for month boundaries.

Primary key choice, and why it's an ADR candidate. Because the client generates the ID, the key is a **UUID.** A purely random UUID (version 4) as a primary key scatters inserts across the index, which hurts locality and cache efficiency as the table grows. **UUIDv7** values are time-ordered, which keeps inserts mostly appending to one end of the index, much like a sequence does. PostgreSQL 18 added a built-in `uuidv7()` function; on older versions you generate it in application code. For a table of a few million rows either works, but you would not choose a pattern you know to be inferior when the alternative is free. *(Verify the exact PostgreSQL version support of your managed provider before relying on the built-in function.)*

Flow 2: View the monthly summary

```
1. App asks:  GET /v1/summary?month=2026-10
2. API authenticates, identifies user
3. Repository runs one SQL aggregate:
      SELECT category_id, SUM(amount_minor)
      FROM expenses
      WHERE user_id = :uid
        AND spent_on >= :month_start AND spent_on < :next_month_start
        AND deleted_at IS NULL
      GROUP BY category_id;
4. API returns totals per category plus a grand total
5. App caches and displays
```

Two design decisions are visible here.

Compute on read, not pre-aggregated. We could keep a separate "monthly totals" table updated on every write. That is faster to read, but now there are **two copies of the truth** that must stay in sync, and a bug in the sync produces exactly the failure NFR-4 forbids: totals that don't match expenses. At ~1,000 rows per user per month, a single indexed aggregate returns in milliseconds. So we compute on read, and keep the option to add pre-aggregation *later, with evidence*. The rule: **don't denormalise until a measurement says you must.**

`spent_on` **as a DATE instead of a timestamp.** In Lecture 1's quick sketch I used `occurred_at TIMESTAMPTZ`. **I am changing that now, deliberately, and I will record why in an ADR** (that's what ADRs are for: changing a decision *visibly*, not silently). Reasoning: the user's intent is "I spent this on the 31st," a *calendar date* in their own life. If we store a UTC timestamp, then an expense logged at 23:30 in India on October 31st is already November 1st in UTC, and month grouping needs time-zone arithmetic on every query, and travelling users could see expenses jump between months. A `DATE` chosen on the phone (default: today in the phone's local time) removes the whole class of problem. The summary query becomes a simple range on a date column, which is index-friendly. We still keep a `created_at TIMESTAMPTZ` (server time) for auditing. **A DBA tip:** keep the column bare in the `WHERE` clause (compare `spent_on` directly to computed boundaries) instead of wrapping it in a function like `date_trunc`, so the index can be used.

The trade-off we accept: we lose time-of-day, which no requirement needs. If a requirement for "what time did I buy this?" appears later, we add a nullable column. Nothing is destroyed.

Flow 3: Sign in and stay signed in

A mobile app has no browser cookie jar in the usual sense, so a common pattern is **short-lived access token + longer-lived refresh token:**

```
1. User sends credentials  →  API verifies (password hash compare / code)
2. API returns:  access token (short life, e.g. ~15 minutes)
                 refresh token (longer life, random, stored HASHED in DB)
3. App stores both in secure storage (Keychain/Keystore)
4. App calls the API with the access token in the Authorization header
5. When the access token expires, app sends the refresh token
6. API checks it against the DB, ISSUES A NEW refresh token and invalidates the
old one
   ("rotation"). If an old refresh token is ever reused → treat as theft,
   revoke that token family and force sign-in.
7. Sign out = revoke the refresh token in the DB
```

Why store refresh tokens in the database? Because it lets you **revoke** them (lost phone, sign out, account deletion). Pure stateless tokens cannot be revoked until they expire, which is acceptable for the short-lived access token and not for the long-lived refresh token.

The sign-in method is not decided yet. In the PRD, open question Q-3 asks whether it will be email + password or a one-time code. A lead does not guess. The design records this as a **proposed** decision (ADR-005 below) with a default and the reason it is blocked. **Important advice:** don't invent your own cryptography or authentication scheme. Use well-established libraries, or consider a managed identity provider, and weigh its cost, lock-in, and the difficulty of migrating password hashes later. *(Check the current maintenance status of any authentication library before you adopt it. This area changes.)*

Flow 4: Delete an account

NFR-8 says: personal data removed from live systems within 30 days, and backups expire within 35 days.

```
1. User confirms deletion in the app
2. API immediately: revokes all tokens, marks the account "pending deletion",
   blocks sign-in, and returns success
3. A scheduled job runs daily: finds accounts pending deletion for > N days
   (a short grace period allows an "I changed my mind" window, if the client
   agrees), then deletes the user's rows (foreign keys with ON DELETE CASCADE
   make this one reliable statement per user), records an audit line with
   no personal data ("account X purged on date Y")
4. Backups: configure retention so that no backup is kept longer than 35 days
```

Look at the **two-phase design.** Phase 1 is immediate and cheap and makes the account unusable. Phase 2 does the heavy deletion on a schedule. This is more robust than a long delete

inside a web request that might time out halfway. And it gives you the compliance story: "access is cut instantly, data is purged within 30 days, backups age out within 35."

Soft delete vs hard delete for expenses. FR-3 says deleted expenses are excluded from totals. I choose **soft delete** (a `deleted_at` timestamp) with a purge job removing rows after a retention window (say 30 days). Reason: it makes an "undo" possible, protects against accidental deletes, and gives future sync features something to reference. The trade-off: every query must remember `deleted_at IS NULL`, and forgetting that is a bug that shows up as wrong totals. Mitigations: use a **partial index** and a single repository function for reads, and add an automated test that deletes an expense and re-checks the totals.

Flow 5: Export to CSV

For v1 the export streams a CSV from a database cursor in the HTTP response. A user has at most thousands of rows, so this is fine. If it ever took longer than a request timeout, the design would change to "generate asynchronously and notify," which needs a job queue. We note the trigger and don't build it now.

Flow 6: Usage events

The app sends events (`app_opened`, `expense_saved`) to the API. They are stored in an append-only `usage_events` table, **with no expense contents, amounts, or notes** (FR-10). The reason for storing them in our own database rather than a third-party analytics service: smaller privacy surface, no extra vendor, and the success metric can be computed with a SQL query. This table can grow faster than others, so we plan for it: simple structure, an index on (user_id, created_at), and a note that partitioning is the option if it grows large.

3.6 Failure handling

A good lead designs around failure from the first day, because in production **failure is not an exception; it's a normal operating condition.** For each place something can fail, I ask three questions: *What does the user see? What happens to the data? How do we find out?*

What fails	What would go wrong	Design response
Phone loses network mid-save	User unsure if saved; might re-tap and duplicate	Client keeps form data on screen; retries with the **same UUID**; server idempotency makes retry safe
Server commits, response lost	Client thinks it failed	Same as above: replay returns the existing row
User taps Save twice	Duplicate expense	Client disables the button during the request *and* the unique constraint blocks duplicates
API process crashes mid-request	Half-written data	Single database **transaction**: all-or-nothing; no partial expense
Database unavailable	All requests fail	Health endpoint; API returns 503 with a clear error; client shows a friendly retry message; monitoring alerts you
Slow query hogs the database	Everyone's latency suffers	`statement_timeout` for the API role; an index on every frequent filter; monitor slow queries with `pg_stat_statements`
Too many connections	New requests fail with connection errors	Bounded connection pool in each API process; **total pool size across all processes stays below the database's** `max_connections` (with headroom for admin and migration sessions); use a pooler like PgBouncer if copies multiply
Deploy while users are active	Dropped requests; mismatched schema	Graceful shutdown; migrations written to be **backward compatible** so old and new code both work during rollout
Bad release	Broken app for everyone	Fast rollback of the API; database changes designed so rolling back code does not require rolling back data (more in Module 6)
Old app versions still installed	New API breaks old clients	Versioned API (`/v1`); only *additive* changes within a version; an endpoint that reports the **minimum supported app version** so you can force an upgrade when necessary

What fails	What would go wrong	Design response
Phone clock is wrong	Bad timestamps	Server sets `created_at`; `spent_on` is user intent and is range-checked
Token expires mid-use	Request fails with 401	Client refreshes once and replays the request; if refresh fails, returns to sign-in
Retry storm	Clients retry instantly during an outage and make it worse	Client retries with **exponential backoff and jitter** (increasing, randomised delays), and **only** for requests that are safe to retry
Error-tracking or analytics service down	App slows or crashes because of a *monitoring* tool	Telemetry fails *open*: it is best-effort and never blocks user actions
Argon2 hashing blocks the server	One slow sign-in stalls all requests on an async server	Run CPU-heavy work (password hashing) in a thread pool or a synchronous endpoint, not directly in an `async def` handler

Three more habits that belong to this topic:

- **A consistent error format.** Pick one structure for all API errors so the client can handle them uniformly. There is a standard for this called *Problem Details for HTTP APIs* (RFC 9457), a good default.
- **Correlation IDs.** Every request gets a `request_id`, which appears in logs and in the error returned to the client. When Meera says "it failed at 6:42," you can find the exact request.
- **Decide what is acceptable to lose.** One API instance and one database instance is a *single point of failure*. For a ten-user pilot with a 99.5% target (about 3.6 hours per month) that is an acceptable, **deliberately accepted** risk, and it is written in the design doc as such. "We didn't think about it" and "we decided to accept it" are very different statements.

DBA connection. Notice how much of this table is about things you already know: transactions, constraints, connection limits, timeouts, slow queries, backward-compatible schema changes. Much of "production-grade backend" is *database discipline applied to the application*. This is your hidden advantage.

3.7 Security

Security is not a layer you add at the end. It is a set of design decisions. The method is a light **threat model.** You ask four questions: *What are we protecting? Who might attack it? How might they? What do we do about it?*

Assets: user credentials, financial records, tokens, the database, backups, the signing keys and other secrets.

Threat actors and scenarios:

Actor	Scenario	Control
Anonymous attacker	Guesses passwords or floods the login	Rate limiting and lockout/backoff on sign-in; slow, salted password hashing (Argon2id is the current common recommendation)
Anonymous attacker	Calls the API directly with crafted data	Server-side validation of everything (Pydantic); parameterised SQL (never string-built queries)
Authenticated user attacking another user	Changes an ID in a request to read someone else's expenses	**Object-level authorisation on every query** (see below)
Person who finds a lost phone	Reads tokens or data	Tokens in Keychain/Keystore, not plain storage; short access-token life; refresh-token revocation on sign-out
Attacker who obtains the app binary	Extracts secrets from the app	**Never put secrets in the mobile app.** Anything shipped in the binary should be treated as public
Insider or leaked credentials	Reads the production database	Least-privilege database roles; no shared admin account; secrets in the platform's secret store, not in the repository
Dependency vulnerability	A compromised or vulnerable package	Pin versions, automated dependency scanning, review before upgrading

The one that beginners miss: broken object-level authorisation (BOLA). It's ranked first in the OWASP API Security Top 10. It happens when an API takes an ID from the client (`GET /expenses/123`) and returns the record without checking that it belongs to the caller. It is extremely common. The defence is structural: **every repository query includes the current user's ID as a filter** (`WHERE id = :id AND user_id = :uid`), so forgetting is hard. A strong

DBA-flavoured addition: **PostgreSQL Row-Level Security (RLS)** as defence in depth, where the database itself refuses to return rows that do not belong to the session's user. It is powerful, but it adds complexity: you must set the user context on each connection or transaction, and it interacts with connection pooling. I'd treat it as an *optional hardening step,* record it as a considered alternative, and decide with eyes open.

Other security design points:

- **TLS everywhere:** client to API, and API to database.
- **Database roles:** the API connects with a role that can read and write data but **cannot alter schema.** A separate role runs migrations. A leaked API credential then cannot drop tables.
- **Encrypted storage and backups:** use the provider's encryption at rest. Field-level encryption of amounts is not justified for v1 (it would break SQL aggregation); revisit with a concrete threat.
- **Logging hygiene:** do not log tokens, passwords, or full request bodies. Amounts and notes are personal financial data; treat them as sensitive in logs too.
- **Data minimisation:** the usage events table stores no financial content.
- **Certificate pinning** in the mobile app is *context-dependent.* It defends against some network attacks but creates an operational hazard (a certificate rotation can lock every installed app out). Not for the MVP; revisit if the threat model changes.
- **Compliance:** financial and personal data may bring legal duties, for example India's Digital Personal Data Protection Act, 2023. **I am not a lawyer;** have the client confirm applicable obligations, and keep the design flexible: data deletion, export, a privacy policy, and data kept in an appropriate region.

3.8 Scalability

The most common mistake in this area is building for a scale you do not have. The second most common is building something that *cannot* scale when the time comes. A lead avoids both with three moves.

Move 1: Know the order of bottlenecks. For this kind of app, the typical order is:

1. A missing index or a bad query (fix with an index or a rewrite).
2. Database connections (fix with a pooler).
3. The single database's CPU or I/O (scale up the instance first; add read replicas for read-heavy workloads later).
4. The single API instance (add more copies; this is easy because the API is stateless).
5. Very large tables, such as `usage_events` (partitioning, archiving).

Move 2: Scale up before you scale out. A bigger machine is the simplest scaling step, and PostgreSQL on modern hardware handles far more than most apps ever need. Complexity (sharding, microservices, caches) comes last.

Move 3: Buy cheap insurance, don't build speculative machinery. The list below costs almost nothing today and saves a rewrite later:

Cheap insurance (do now)	Speculative construction (don't do now)
Stateless API	Kubernetes cluster
`/v1` API versioning	Message broker (Kafka, etc.)
UUID keys generated by the client	Redis cache for data that is already fast
`user_id` on every user-owned table	Read replicas
Pagination on every list endpoint from day one (prefer *cursor-based* pagination for lists that grow)	Sharding
Indexes matched to real queries	Splitting into services
Database migrations from the first day	A custom job-queue framework
Metrics that tell you when a trigger is hit	Autoscaling rules

Write the triggers down. In the design document I include a "scale triggers" table: a measured signal and the action it prompts. For example: "p95 save latency above 300 ms for a week" → investigate queries and indexes first; "database CPU above 60% sustained" → scale the instance up; "connection errors" → introduce a pooler; "weekly active users approaching 1,000" → review the design and the NFR numbers, which were chosen for a 10-user pilot. A trigger turns a vague worry into an agreed action.

A DBA reminder on indexes: every index speeds some reads and slows every write, and takes space. Start with the indexes that match the known queries, check them with `EXPLAIN`, and remove the ones that are never used.

3.9 Operational constraints

An architecture that cannot be operated is not production-grade, no matter how elegant the diagram. For each decision, ask "who runs this, and what do they do when it breaks?"

Hosting. The options range from a single virtual machine with Docker Compose (cheap, you own everything, including patching and backups), to a Platform-as-a-Service (you deploy a container; the platform manages servers), to a cloud container service with a managed database. For a solo engineer with a 99.5% target, I choose **a PaaS or a managed container service for the API and a managed PostgreSQL service for the database.** Reasoning: it shrinks the surface you are responsible for.

Why a managed PostgreSQL even though you're a DBA? You *could* run PostgreSQL yourself, and your skills are real. But for a solo lead, the scarce resource is attention. Running your own Postgres means you own OS patching, backup configuration, failover, and monitoring, on top of everything else. A managed service handles backups, point-in-time recovery (PITR), patching, and monitoring hooks. You keep what matters: schema design, indexes, query tuning, roles, and the restore drill. *Decision-making rule: spend your effort where it creates unique value.* That said, if cost or control demands self-hosting, it's a valid choice; record it as such.

Region and data residency. The users and the client are in India, so choose an Indian region (for example, Mumbai on AWS or GCP) unless a requirement says otherwise. This reduces latency and eases compliance conversations.

Backups and restore. The NFR says RPO 24 h and RTO 8 h, and backups must age out within 35 days. Configure automated backups plus PITR with a retention at or below 35 days *(check your provider's limits; for example, Amazon RDS permits up to 35 days of automated backup retention)*. Then do the part people skip: **actually perform a restore into a separate database and verify the data before launch.** The old DBA saying applies: *an untested backup is a hope, not a backup.* The PRD already requires one restore drill. Record how long it took, because that number is your real RTO.

Environments. Local (Docker Compose with PostgreSQL), staging (a small copy of production), and production. Staging exists to rehearse deployments and migrations. **Never copy production data into staging without anonymising it;** it's personal financial data.

Deployment and migrations. Build the API into a container image in a CI pipeline, run the tests, and deploy. Schema changes are managed with a migration tool (Alembic is the usual companion to SQLAlchemy) and run as a **separate step** before the new code is released. Write migrations so that old and new code both work against the new schema (the *expand and contract* approach, covered in Module 6). Two PostgreSQL details worth remembering: create indexes on busy tables with `CREATE INDEX CONCURRENTLY` so writes aren't blocked, and remember that adding a column with a constant default is a quick metadata change on modern versions but that other table rewrites can lock the table.

Observability. The minimum for a production system:

- **Logs:** structured (JSON) with `request_id`, user id (not personal content), route, status, duration.
- **Errors:** an error-tracking service for the API and for the mobile app (crash reports).
- **Metrics:** request rate, error rate, latency percentiles (p50/p95/p99), database connections, disk, CPU, slow queries.
- **Uptime:** an external check on a health endpoint.
- **Alerts:** only for things that need human action, routed to you. Too many alerts and you will start ignoring them.

Mobile release constraints. You cannot instantly roll back or update an app that is installed on phones: store reviews take time and users update slowly. Two consequences for the architecture: (1) the API must tolerate **old app versions** for a long time (hence versioning and additive changes), and (2) if you use over-the-air update mechanisms (for example those offered with Expo) they can update the JavaScript layer quickly, but not native code, and store policies apply. Plan for a **forced-upgrade path** from the very first release; it's a tiny feature that is painful to add after the fact because old versions can't be forced to learn about it.

Cost. Put a monthly cost estimate in the design (hosting, database, error tracking, store fees). Check the providers' current pricing yourself, since prices change. Cost is an operational constraint, and the client should see it. A design the client cannot afford to run is not a design.

The "bus factor" question. If you are unavailable for two weeks, can someone else deploy, restore, and fix a problem? A README, a runbook for the three most likely incidents, and infrastructure described as code (or at least documented) are part of the architecture.

3.10 From reasoning to documents: the TDD and the ADR

We now have a lot of reasoning. How much of it goes on paper, and in what form? There are two document types that work together.

The Technical Design Document (TDD) describes **how the system will be built:** the architecture, components, data, flows, failure handling, security, and operations. It is for the people building and operating the system, and for the reviewer who must be able to say "yes, this will meet the requirements." It is a *snapshot of the current design.*

The Architecture Decision Record (ADR) captures **one significant decision:** the context, the options considered, the choice, and the consequences. It is a *historical log.* The commonly used format (from Michael Nygard) has these parts: **Title, Status, Context, Decision, Consequences.** Many teams add **Alternatives considered** and **Revisit if.** An ADR is **never edited to change history.** If the decision changes, you write a *new* ADR that supersedes the old one and update the old one's status to "Superseded by ADR-0xx". That preserves *why* you once believed something.

How they relate. The TDD says *what the design is*. The ADRs say *why*, for the decisions that were hard, contested, or expensive to reverse. The TDD links to the ADRs rather than repeating them.

When to write an ADR. Write one when the decision is (a) hard to reverse, (b) had real alternatives, or (c) someone later will ask "why on earth did we do it this way?" Do **not** write one for "which date library" or "tabs vs spaces." If you write 60 ADRs for a ten-user pilot, nobody will read them. For Spendwise I'd expect around five to eight ADRs in the first month.

Part 4 — Practical demonstration: from the Spendwise PRD to a design

Now let me do the work in sequence, showing a weak approach first and then a stronger one.

4.1 The weak approach

A first draft from an inexperienced developer might look like this:

```
Tech Stack:
- React Native (mobile)
- FastAPI (backend)
- PostgreSQL (database)
- Redis (caching)
- Kafka (events)
- Kubernetes (deployment)
- Microservices: auth-service, expense-service, report-service
- JWT for authentication

Notes: The app will be scalable and secure.
```

I want you to look at what is wrong, item by item:

1. **No reasons.** Nothing says *why* Redis, Kafka, or Kubernetes. This is a list of keywords, not a design.
2. **No link to requirements.** Which NFR does Kafka serve? None. For ten users, it adds cost and failure modes.
3. **Resume-driven development.** The technologies chosen are impressive, not necessary.
4. **No boundaries or data flows.** Who owns the data? What happens when a request fails halfway?
5. **No failure handling.** What if Redis and the database disagree about a total?
6. **"Scalable and secure"** are adjectives. We banned those in Module 2.
7. **No operational view.** Who deploys Kubernetes? You? At 3 a.m.?
8. **No alternatives or revisit conditions.** A decision nobody can challenge.

4.2 The stronger approach, step by step

Step 1: Extract the ASRs from the PRD.

I read the NFRs and exclusions and ask for each: *"If this changed, would the structure change?"*

ASR	Source	Why it's architecturally significant
ASR-1 Exactly-once expense creation on retry	NFR-5	Affects API contract, ID strategy, DB constraints
ASR-2 Totals always equal sum of undeleted expenses	NFR-4	Argues for a single source of truth and compute-on-read
ASR-3 Users see only their own data	NFR-7, FR-1	Affects every query and possibly DB-level controls
ASR-4 RPO 24 h / RTO 8 h; backups expire ⩽ 35 days	NFR-6, NFR-8	Affects database hosting and backup configuration
ASR-5 99.5% availability	NFR-3	Affects deployment (graceful restarts, health checks)
ASR-6 Personal data deletion within 30 days	NFR-8, FR-9	Affects data model (cascades), job design
ASR-7 Old app versions must keep working	Mobile reality + NFR-9	Affects API versioning and forced-upgrade path
ASR-8 Usage events without financial content	FR-10	Affects event schema and privacy design
Constraint C-1	Solo engineer, small budget	Affects every operational choice
Constraint C-2	No offline, no bank sync, no sharing (v1)	Simplifies; but protect with cheap insurance

Notice that "p95 500 ms" is *not* on the list. It matters, but my estimate shows it is easy to meet with sound queries, so it doesn't drive the structure.

Step 2: Load and size. The back-of-envelope in section 3.2: even at 1,000 users, a few writes per second and about a gigabyte a year.

Step 3: Generate and evaluate alternatives. The comparison table in section 3.3. Choice: modular monolith, managed PostgreSQL, stateless API, PaaS or managed containers.

Step 4: Draw the boundaries. The diagram and the seven rules in section 3.4.

Step 5: Trace the key flows. The six flows in section 3.5, with the failure table in 3.6.

Step 6: Security and operations pass. Threat table and operational list.

Step 7: Record the decisions that were hard or expensive to reverse as ADRs.

Step 8: Assemble the TDD, link it to the PRD requirement IDs, and put it up for **review** (by the client for the parts they can judge, and ideally by another engineer for the rest).

Step 9: Read it as the person who will build it and as the person who will be woken at 3 a.m., and fix what they would struggle with.

4.3 The sample Technical Design Document

Here is a realistic TDD for the Spendwise MVP. It is compact, but every section does work. Notice how each section points back to a requirement ID.

markdown

```
# Technical Design: Spendwise MVP
**Version:** 0.1 (Draft)   **Date:** 2026-10-14   **Author:** Tech Lead
**Status:** Draft — for review   **Implements:** PRD v0.1
**Related ADRs:** 001, 002, 003, 004, 005 (proposed)

## 1. Purpose and scope
Describes how the MVP defined in PRD v0.1 will be built, run, and secured.
Out of scope: detailed ERD and API contract (separate documents, to follow);
features excluded in PRD §5.

## 2. Architecturally significant requirements
| ID | Requirement | PRD ref |
|---|---|---|
| ASR-1 | Exactly one expense stored on retry or double-tap | NFR-5 |
| ASR-2 | Totals equal sum of undeleted expenses | NFR-4 |
| ASR-3 | User can access only own data | NFR-7 |
| ASR-4 | RPO 24 h, RTO 8 h; backups expire <= 35 days | NFR-6, NFR-8 |
| ASR-5 | 99.5% monthly availability | NFR-3 |
| ASR-6 | Personal data deleted within 30 days of request | NFR-8, FR-9 |
| ASR-7 | Old app versions keep working | NFR-9 |
| ASR-8 | Usage events contain no financial content | FR-10 |

## 3. Constraints and assumptions
- One engineer; ~8 weeks; small budget; iOS + Android.
- Load: ~10 users now; design headroom to ~1,000 (estimate: <= 3 writes/s peak,
  ~1 GB/year). NFR numbers valid for pilot; revisit before >1,000 users.
- Assumptions: INR only (A2); no bank import (A1); single owner per record (A3);
  users mostly online (A4).
- Open: sign-in method (Q-3) → ADR-005 proposed.

## 4. Architecture overview
Mobile app (React Native, TypeScript) → HTTPS/JSON `/v1` → stateless FastAPI
service → PostgreSQL (managed, India region, PITR). A scheduled job (same
codebase) purges deleted accounts and expenses. Logs, error tracking, and uptime
monitoring are side channels. [Diagram: see §4 figure in repo docs/architecture.]
Style: modular monolith (ADR-001).

## 5. Components and responsibilities
| Component | Owns | Must never |
|---|---|---|
| Mobile app | UI, local form validation, secure token storage, UUID generation, r
```

```
| API: auth module | Credentials, tokens, sessions | Return password material |
| API: expenses module | `expenses` table, save/edit/delete rules | Be bypassed by
| API: summaries module | Read-only aggregates | Store derived totals (compute on
| API: account module | Account deletion workflow | Delete synchronously inside a
| API: events module | `usage_events` | Accept amounts, categories, or notes |
| Scheduled job | Purge of pending deletions and old soft-deleted rows | Run insid
| PostgreSQL | Durable data, constraints, backups | Be reachable from the internet

Internal layering per module: router → service → repository. Dependencies point
downward only. Every repository query on user data filters by the caller's user id

## 6. Data design (summary; full ERD to follow)
Entities: users, categories (fixed list, stored as data), expenses,
refresh_tokens (hashed), usage_events, deletion_requests.
Key decisions:
- Money: `amount_minor BIGINT` (paise) + `currency_code CHAR(3)`; no floating poir
- Date: `spent_on DATE` (user's calendar date) + `created_at TIMESTAMPTZ` (server)
- Keys: UUID (client-generated, time-ordered) for expenses. (ADR-003)
- Soft delete via `deleted_at`; purge job removes after retention window.
- Constraints enforce the rules: NOT NULL, CHECK on amount range, FKs with
  ON DELETE CASCADE from users.

Draft DDL for the central table:

    CREATE TABLE expenses (
      id             UUID PRIMARY KEY,
      user_id        UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
      category_id    SMALLINT NOT NULL REFERENCES categories(id),
      amount_minor   BIGINT NOT NULL
                     CHECK (amount_minor > 0 AND amount_minor <= 1000000000),
      currency_code CHAR(3) NOT NULL DEFAULT 'INR',
      spent_on       DATE NOT NULL,
      note           TEXT CHECK (char_length(note) <= 200),
      created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
      updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
      deleted_at     TIMESTAMPTZ
    );
    CREATE INDEX expenses_user_spent_on_idx
      ON expenses (user_id, spent_on) WHERE deleted_at IS NULL;

(The upper bound 1,000,000,000 paise equals 10,000,000.00 INR, matching PRD US-2 A
```

7. API overview (full contract to follow)
REST over HTTPS, JSON, `/v1` prefix, OpenAPI generated by FastAPI; TypeScript types generated for the client. Bearer access tokens. Lists are paginated. Errors use a single problem-details format with a request id.
Endpoints (summary): auth (sign-in, refresh, sign-out); POST/PATCH/DELETE expenses; GET expenses (paginated, filter by month and category); GET summary; GET export; POST usage events; DELETE account; GET meta (min supported app versior

8. Key flows
Save expense, view summary, sign-in/refresh, delete account: see lecture diagrams; each flow lists failure behaviour in §9.
Save expense: client UUID → POST → validate → transaction:
INSERT ... ON CONFLICT (id) DO NOTHING → if conflict: read back and return existing if same user and same payload, else error → commit → 201 or 200.

9. Failure modes and handling
Failure	Handling	Verified by
Network loss / lost response	Client retries with same UUID; idempotent insert
Double tap	Button disabled in flight; unique constraint	Automated test
API crash mid-request	Single transaction	Fault-injection test
Database unavailable	503 + friendly message; health check; alert	Manual dril
Slow query	statement_timeout; slow-query monitoring	Load test
Connection exhaustion	Bounded pools; total below max_connections; pooler if ne
Old app versions	Versioned API; additive changes; minimum-version endpoint	C
Bad deploy	Rollback runbook; backward-compatible migrations	Staging rehearsa
Single point of failure (1 API, 1 DB)	**Accepted risk** for pilot; revisit at

10. Security design
Threats: credential attacks, direct API abuse, cross-user access (BOLA), lost device, extracted app secrets, leaked credentials, vulnerable dependencies.
Controls: TLS client→API and API→DB; Argon2id password hashing; rate limits on sign-in; Pydantic validation and parameterised SQL; user-id filter on every query (+ evaluate Row-Level Security as hardening); tokens in Keychain/Keystore; refresh-token rotation and revocation; no secrets in app or repo; separate DB roles for app and migrations; no tokens or financial content in logs.
Compliance: privacy policy, deletion flow, region choice; client to confirm legal obligations (not legal advice).

11. Performance and scalability
Estimate: <= 3 writes/s, ~12 reads/s peak at 1,000 users; ~3.65M rows/year.
Target NFR-1 is expected to be met with indexed queries. No cache, queue, or

```
read replica in v1.
Scale triggers:
| Signal | Action |
|---|---|
| p95 save latency > 300 ms for 7 days | Inspect queries and indexes first |
| DB CPU > 60% sustained | Scale instance up |
| Connection errors | Add pooler (PgBouncer) |
| Weekly active users near 1,000 | Review design and NFR numbers |
| usage_events very large | Partition or archive |

## 12. Operations
Hosting: PaaS/managed containers for API; managed PostgreSQL in India region.
Environments: local, staging, production (no unmasked prod data in staging).
CI: tests and image build on every change; deploy on approval.
Migrations: Alembic, separate step, backward compatible.
Backups: automated + PITR, retention <= 35 days; restore drill before launch
(measure real RTO).
Observability: structured logs with request id; error tracking (API and app);
metrics (rate, errors, p95, DB connections, disk, slow queries); uptime check;
alerts only for actionable events.
Mobile: forced-upgrade path from v1.0; OTA updates for JS-only fixes where
allowed.
Cost: monthly estimate to be added from current provider pricing.
Continuity: README and runbook for deploy, restore, and top three incidents.

## 13. Risks and open questions
- R1: Soft-delete filter forgotten → wrong totals. Mitigation: single read path +
- R2: App-store review delays a critical fix. Mitigation: API backward compatibili
- R3: Single instance/DB outage > 3.6 h/month. Mitigation: monitoring; multi-AZ DB
  option costed.
- Q-3 Sign-in method (owner: Meera + Lead) blocks ADR-005.
- Q-4 Region and provider choice to be confirmed against client's cost ceiling.

## 14. Change log
| Version | Date | Change |
|---|---|---|
| 0.1 | 2026-10-14 | First draft |
```

4.4 The Architecture Decision Records

Here are four of the ADRs, written in full, so you can see what "the right level of detail" looks like. Each is short. That is deliberate.

markdown

```
# ADR-001: Use a modular monolith (one API service, one database)
**Status:** Accepted    **Date:** 2026-10-14    **Decided by:** Tech Lead (client

## Context
Spendwise MVP: ~10 pilot users, designed for headroom to ~1,000. One engineer.
Key drivers: data correctness (ASR-1, ASR-2), operability by a solo engineer (C-1)
account deletion control (ASR-6). Estimated peak load <= 3 writes/s.

## Options considered
A. Modular monolith — chosen
B. Microservices — rejected: no organisational or scaling need; adds distributed
   failure modes and many deployments
C. Serverless functions + managed DB — viable; rejected for now: harder local
   debugging, platform retries can double-invoke handlers, needs a connection
   pooler for PostgreSQL
D. Backend-as-a-Service — viable; rejected for now: business rules for money and
   deletion preferred in reviewable code; higher exit cost
E. Local-first — rejected: offline excluded in v1; sync conflict complexity

## Decision
One FastAPI service with internal modules (auth, expenses, summaries, exports,
account, events) and layers router → service → repository, backed by one
PostgreSQL database. Background work runs as a separate scheduled entry point
from the same codebase.

## Consequences
+ One place for rules and transactions; simple to deploy and debug.
+ Module seams allow later extraction.
- Single deployable: a defect can affect all modules; discipline is needed to keep
  module boundaries (enforced by code review and import rules).
- Single point of failure accepted for the pilot (see TDD §9).

## Revisit if
Multiple teams need independent releases; a feature needs very different scaling
(e.g. image processing); offline becomes a Must; operations burden exceeds capacit
```

markdown

```
# ADR-002: Store money as integer minor units; store expense date as a DATE
**Status:** Accepted    **Date:** 2026-10-14
**Supersedes:** the `occurred_at TIMESTAMPTZ` sketch in the Module 1 working notes

## Context
Amounts must match bank statements exactly (NFR-4). Month totals must be correct
for users in their own calendar (US-5 AC4: an expense at 23:59 on the last day
counts in that month). A UTC timestamp makes month grouping depend on time-zone
arithmetic and on where the user happens to be.

## Options considered
Money: FLOAT (rejected: inexact); NUMERIC(12,2) (acceptable: exact, readable,
fixed scale); BIGINT minor units (chosen: exact, compact, explicit currency).
Date: TIMESTAMPTZ (rejected: month boundaries need per-user time zone logic);
DATE chosen by the client (chosen).

## Decision
`amount_minor BIGINT` with `currency_code CHAR(3)`, and `spent_on DATE` holding th
user's calendar date; `created_at TIMESTAMPTZ` set by the server for audit.

## Consequences
+ Exact arithmetic; month totals are a simple indexed range query.
+ Travel and time-zone changes cannot move an expense between months.
- Conversion at the UI boundary (divide by 100 to display); currencies with a
  different number of minor units need care when added.
- Time of day is not recorded; a future need adds a nullable column.

## Revisit if
Multi-currency or crypto is added; a requirement for time of purchase appears.
```

markdown

```
# ADR-003: Client-generated UUID with idempotent create
**Status:** Accepted    **Date:** 2026-10-14

## Context
NFR-5: a retry or double tap must create exactly one expense. The response to a
successful save can be lost on a mobile network.

## Options considered
A. Server-generated bigint id — rejected: a retry looks like a new request.
B. Separate Idempotency-Key header with a key store — viable; more moving parts.
C. PUT /expenses/{uuid} (create-or-replace) — naturally idempotent; rejected: a
   retry with different content would silently overwrite.
D. POST with client-generated UUID as primary key; INSERT ... ON CONFLICT DO
   NOTHING, then read back — chosen.

## Decision
The client generates a UUID (time-ordered, v7) for each new expense. POST /v1/expe
includes it. On conflict: same user and same content → return the existing row (20
different user or different content → error. Database primary key is the final gua

## Consequences
+ Retry-safe without extra infrastructure; also prepares for offline in future.
+ Time-ordered UUIDs keep index inserts append-like.
- UUIDs are larger than bigints (16 vs 8 bytes) in indexes and foreign keys.
- Server must validate the UUID format and never trust client IDs for authorisatic

## Revisit if
Requests need cross-resource idempotency (e.g. multi-step operations).
```

markdown

```
# ADR-005: Authentication approach
**Status:** Proposed — blocked on PRD open question Q-3 (sign-in method)
**Date:** 2026-10-14

## Context
Sign-in method (email + password vs one-time code) is not yet decided by the clier
Financial data raises the cost of getting authentication wrong. One engineer.

## Options considered
A. Own implementation with well-established libraries (Argon2id, signed tokens,
   rotating refresh tokens stored hashed)
B. Managed identity provider (hosted sign-in, token issuance, recovery flows)

## Decision (provisional)
Prefer B if the client accepts the recurring cost and the provider's data-residenc
terms; otherwise A. Do not hand-roll cryptography in either case.
Final decision after Q-3 and a short cost/lock-in check.

## Consequences (if B)
+ Less security-critical code to own; account recovery flows provided.
- Vendor lock-in; migration of users later is harder; cost grows with users.
## Consequences (if A)
+ Full control; no per-user fees.
- More security responsibility; maintenance of the auth module.

## Revisit if
Number of users or requirements (social sign-in, multi-factor) change.
```

What to notice about these documents:

1. **Each ADR is under a page.** Longer, and nobody reads them.
2. **Alternatives are real and fairly described,** including options that are *viable* but not chosen. That is what makes the decision challengeable and credible.
3. **Consequences include the negatives.** An ADR that lists only benefits is marketing.
4. **ADR-002 openly supersedes an earlier sketch.** The change is visible and reasoned, never silent.
5. **ADR-005 is "Proposed."** It is honest about being blocked on an open question and states the default. A lead never fakes certainty.
6. **The TDD links to ADRs** instead of re-explaining them, and links to requirement IDs so you can trace *why* each design element exists.

Part 5 — Documentation: what the lead records, and how much

Document	Purpose	Detail level	Update rule
TDD	Current design; basis for review and onboarding	4 to 8 pages for an MVP; each section short and linked to requirement IDs	Update when the design changes; keep a change log
ADRs	The *why* behind hard-to-reverse decisions	Under one page each; Context, Options, Decision, Consequences, Revisit-if	Never rewrite history; supersede with a new ADR
Architecture diagrams	Shared mental model	Two levels are usually enough: *system context* (who and what talks to the system) and *containers* (the deployable parts). This follows the useful "C4 model" idea of showing different zoom levels	Keep as text-based diagrams in the repository (for example Mermaid or PlantUML) so they change with the code
Runbook	What to do when it breaks	One page for the top few incidents	Update after every real incident (Module 6)
Assumption and risk log	Visible uncertainty	Continue from Modules 1 and 2	Review at each planning session

Where to keep them. In the repository, in a `docs/` folder, versioned with the code. Documents that live in someone's email or chat drift and die.

Review before build. Walk through the design with a second person: the client for what they can judge (cost, region, exclusions, what "accepted risk" means), and another engineer, if available, for the technical content. A design review is far cheaper than a rewrite. Even a self-review the next morning, using the checklist "would this satisfy every ASR, and what is the first thing that will break?", finds problems.

Practices that keep documentation useful

- **Write the design before building the expensive parts, not for the whole system.** Detail for the hard-to-reverse parts, light touch for the rest.
- **Trace everything.** ASR → design element → test. If a design element has no requirement behind it, ask whether it should exist.
- **Record accepted risks explicitly.** "Single instance: accepted for the pilot."

- **Separate fact from preference.** "PostgreSQL's default isolation level is READ COMMITTED" is fact. "I prefer UUIDv7" is a decision with reasons.

Anti-patterns

- **Architecture astronaut documents:** many diagrams and layers, no link to a real requirement.
- **Diagrams that were accurate once.** An outdated diagram is worse than none, because people trust it.
- **ADRs written after the fact to justify choices already made.** They lack real alternatives.
- **The "template with empty sections."** If a section has no content, delete it, or write "none, because...".
- **One giant document for everything.** Split into TDD, ERD, API contract, and ADRs.

Part 6 — Trade-offs and failure scenarios

I'll label items **[Established]** where the practice is widely agreed and **[Context]** where your judgment decides.

6.1 Microservices too early [Established]

Scenario. A solo developer splits Spendwise into three services "so it scales." Now saving an expense involves two network calls, a failure between them can leave an expense saved with no total updated, and every release requires deploying three things in the right order. A bug that was a stack trace becomes a mystery between services. **Cost:** weeks of effort and a worse system. **Rule of thumb:** distribute only when you must.

6.2 The monolith with no boundaries [Established]

Scenario. All code in one folder, with SQL in route handlers and business rules copied across endpoints. A month later "exclude deleted expenses" is applied in five places, and the sixth, the CSV export, forgets it. The export includes expenses users deleted, which is also a privacy problem. **Fix:** one repository read path and a modular structure, which is why rules 4 and 5 exist.

6.3 Trusting the client [Established]

Scenario. The app validates amounts, so the API doesn't. Someone scripts requests with negative amounts and a different user's ID. **Fix:** Rule 2, plus database CHECK constraints, plus object-level authorisation.

6.4 BaaS: speed now, constraints later [Context]

BaaS gives speed and low operations. The costs: business rules expressed in platform-specific policies, harder automated testing, platform limits, and exit cost. For a throwaway prototype or

a client who wants speed above all, it may be the right call. Choose it *knowingly*.

6.5 Serverless with PostgreSQL [Context]

Many short-lived function instances can each open database connections and exhaust `max_connections`, so you need a pooler or a database proxy. Cold starts add latency. Platform retries can invoke a function twice, which makes idempotency (ADR-003) even more important. Fine when the workload is spiky and the team knows the platform; extra complexity otherwise.

6.6 Premature caching [Established]

Scenario. Someone adds Redis to cache monthly totals "for speed." A user deletes an expense and the cached total still shows the old amount for five minutes, violating NFR-4. Now you have an invalidation problem, which is famously hard, to solve a performance problem that didn't exist. **Rule:** measure, then cache, and prefer caching data where staleness is harmless.

6.7 Pre-aggregated totals [Context]

Maintaining a separate totals table is a legitimate optimisation at large scale. At our scale it adds a second source of truth and a class of bugs. The decision depends on measured query time, and it can be added later without redesign if reads are already behind one repository function.

6.8 No API versioning [Established]

Scenario. You rename a field in the response. New API deployed. Everyone on old app versions sees a broken screen, and you cannot fix their phones. **Fix:** `/v1`, additive-only changes, minimum-version check.

6.9 Overbuilding the insurance [Context]

Lecture 1's rule applies: cheap insurance, not speculative construction. Adding a `currency_code` column is insurance. Building currency conversion is speculation. Using UUIDs from the client is insurance. Building a complete offline-sync engine is speculation.

6.10 Accepted single point of failure [Context]

One API copy and one database instance means any failure is an outage. For a pilot with a 99.5% target, that can be an acceptable and cheap choice, *if* it is written down, monitored, and the cost of removing it is known (for example a multi-zone database option). If Meera later promises Spendwise to paying customers, the decision must be revisited; that's the "revisit if."

6.11 Self-hosted vs managed database [Context]

Self-hosting gives control and possibly lower cost at scale. Managed saves operational attention and provides tested backups, PITR, and patching. For a solo lead and a small pilot,

managed usually wins. For a team with strong DBA capacity and special requirements, self-hosting can win. Neither is universally right.

6.12 Database-level security: RLS [Context]

Row-Level Security adds a strong second layer against BOLA, but complicates connection handling and debugging, and a mistake in a policy can silently hide data. Use it when the threat justifies the cost; always keep application-level filtering as well.

6.13 Migrations that lock production [Established]

A migration that rewrites a large table or builds an index without `CONCURRENTLY` can block writes. With ten users nobody notices. With a thousand, a lunch-time migration becomes an outage. Build the habit now, on a small table, when the stakes are low.

Part 7 — Hands-on exercise

You get to be the lead. This exercise continues from Module 2's exercise: **Release 1.1: Monthly category budgets with alerts.** I'm giving you the requirements; **you design how to build them** within the existing architecture.

Inputs (from the Module 2 reference solution)

- **FR-B1** Set, change, remove a monthly limit per category (one per category, amount > 0).
- **FR-B2** Monthly summary shows spent, limit, remaining, percent used per limited category.
- **FR-B3** When saving/editing/deleting an expense causes a category's spending to **reach or exceed 80% or 100%** of its limit for the first time that month, the user sees an alert.
- **FR-B4** Each threshold alert at most once per category per calendar month; alerts are not withdrawn when spending drops.
- **FR-B5** Calendar month; no rollover.
- **NFR-B1** Budget status always equals the sum of undeleted expenses in the category for the month.
- **NFR-B2** The save response includes updated budget status, p95 ⩽ 500 ms at 50 concurrent users.
- **NFR-B5** A retried save does not create duplicate alerts or expenses.
- **Excluded:** remote push notifications (in-app alert only), rollover, shared budgets, forecasting, blocking.
- **Open from the PRD:** if one expense crosses both thresholds, show one alert for the highest threshold. If a limit is lowered below current spending, no alert, status shows "over limit."

Tasks (about 90 minutes)

1. **ASRs.** List the architecturally significant requirements for this release, and say which existing ADRs they touch.
2. **Questions.** Write eight questions you would ask (or answer for yourself) before designing, grouped by the eight categories in section 3.2. Mark the two that most affect the design.
3. **Alternatives.** There are at least three places the alert logic could live. Describe **three** alternatives (including one that is *wrong* and explain why), compare them against the ASRs in a table, and choose one.
4. **Data flow.** Write the step-by-step flow for "user saves an expense that crosses 80% of a budget," including the database transaction and the response. Then, for **every step,** write what can fail and what happens.
5. **Data design.** Draft the tables needed (columns, keys, constraints, and the indexes you would start with). Explain which constraint guarantees "at most one alert per threshold per category per month," even if two requests arrive simultaneously.
6. **Concurrency and retries.** Describe what happens if: (a) two expenses for the same category arrive at the same instant on two API copies; (b) the first save commits but the response is lost and the client retries. *Hint: think about what the retry response must contain so the user doesn't lose the alert.*
7. **Failure and security table.** At least six failure modes and two security risks specific to budgets, each with a design response.
8. **Scale triggers.** Three measurable signals and the action each triggers.
9. **ADR.** Write **ADR-006** for your main design decision, with alternatives, consequences (including negatives), and a revisit-if.

What I'll look for (so you know the standard)

- Did you notice that this is a **write-path** design problem, not a screen problem?
- Did you reason about concurrency and retries rather than only the happy path?
- Is each decision tied to a requirement and to a failure it prevents?
- Did you avoid speculative machinery (queues, caches, new services)?
- Does the ADR contain a real alternative, honest negatives, and a revisit condition?

Part 8 — Review of a typical attempt

Since this lecture is one-way, I'll review the kind of answer a learner at your level commonly writes. Notice how the review gives hints before corrections.

8.1 A typical first attempt (call the learner "Sam")

> **Alternatives:** (1) Create a `budget-service` and a `notification-service` connected by Kafka. (2) Store budgets in Redis so lookups are fast. (3) Let the app calculate alerts from the data it already has. **Choice:** Microservices, because it is scalable. **Flow:** User saves expense → API stores it → budget-service checks the budget → notification-service sends the alert. **Failures:** Handle errors properly. **Data:** `budgets(id, user_id, category, limit)`; `alerts(id, message)`. **Security:** Use JWT. **Concurrency:** Not likely to be a problem. **ADR:** None.

8.2 The review

What Sam did well.

- Sam listed three genuinely different alternatives, including client-side calculation. Listing alternatives is already better than most first attempts.
- Sam understood that an alert is separate from the expense, and thought about a distinct `alerts` table, which is a good instinct.
- Sam noticed that notifications are a concern worth naming.

Gap 1: the choice is justified by an adjective. "Because it is scalable." Load estimate: a few writes per second. *Hint:* what does the load estimate in section 3.2 say about whether scale is a problem? *Correction:* the drivers are correctness and retry safety, which are easiest to meet when the expense and its alert are written in **one database transaction.** Splitting into services makes that transaction impossible without extra machinery.

Gap 2: the flow has a hole you could drive a truck through. If the expense is saved and then the `budget-service` call fails, the user has saved an expense and **never gets the alert.** If the alert is sent and the expense write fails, they get an alert for an expense that doesn't exist. *Hint:* what single mechanism did we use in Flow 1 to make "all or nothing" true? *Correction:* do both in one transaction.

Gap 3: Redis for budgets. Budgets are tiny and rarely read compared with their importance. Putting them in Redis creates a second source of truth that can disagree with PostgreSQL, violating the spirit of NFR-B1. *Hint:* what happens if Redis restarts or lags? *Correction:* keep budgets in PostgreSQL, keyed by (user, category).

Gap 4: client-side calculation. The client is untrusted, may have stale data, and may be one of two devices. And what if the client crashes after saving but before computing? *Hint:* Rule 2. *Correction:* evaluate on the server, return the result in the save response.

Gap 5: "Handle errors properly" is not a failure analysis. *Hint:* take each step and ask what the user sees and what the data looks like. *Correction:* see the failure table in the reference.

Gap 6: "Concurrency is not likely to be a problem." Two devices, a double tap, or a retry all produce concurrent or repeated requests, and in a finance app "unlikely" still happens to *someone*. *Hint:* what stops two simultaneous requests from both deciding "I crossed 80% first" and both creating an alert? *Correction:* a unique constraint plus a row lock (reference solution).

Gap 7: security is "use JWT." JWT is a token format, not a security design. *Hint:* what stops user A from reading or changing user B's budget by changing an ID? *Correction:* user-id filtering on every query and a composite key that includes the user.

Gap 8: no ADR, no revisit conditions. Decisions without records get re-argued or silently changed.

Overall verdict. Sam reached for technology keywords instead of analysing the write path. The fix is the habit from this lecture: **find the drivers, trace the flow, ask what fails at each step, and choose the simplest structure that satisfies the drivers.**

Part 8B — Reference solution to the exercise

Try to make your own attempt before reading this.

1. ASRs

ASR	Source	Touches
A-1 Alert state and expense must change atomically (no expense without its alert, no alert without its expense)	FR-B3, NFR-B1	ADR-001 (single DB, one transaction)
A-2 At most one alert per threshold, per category, per month, even under concurrency	FR-B4	New ADR-006
A-3 A retried save must not create duplicate alerts, **and must not lose the alert**	NFR-B5	ADR-003 (idempotent create)
A-4 Status always equals the sum of undeleted expenses	NFR-B1	ADR-002 (money, date), compute-on-read
A-5 Save response includes budget status within p95 500 ms	NFR-B2	Indexing, no extra network hops
A-6 Old app versions keep working	ASR-7	Additive response fields
A-7 Budgets and alerts deleted with the account	FR-9, NFR-8	Cascade deletes

2. Questions (grouped; the two that matter most are starred)

1. **Drivers:** ★ Must an alert be guaranteed to reach the user after a retry? (Yes, per NFR-B5; this shapes the data model.)
2. **Load:** How many budgets per user? (At most one per category: about 8.)
3. **Data:** ★ What defines "this month" for the user? (With `spent_on` as a DATE, the month comes from the expense's date, not a time zone calculation.)
4. **Data:** When a limit is changed after alerts fired, may an alert fire again? (The PRD says at most once per category per month; flag as an open question to confirm with Meera.)
5. **Change:** Will push notifications come soon? (Probably; design so the alert record can feed a worker later.)
6. **Failure:** What should the user see if the alert evaluation fails but the expense could be saved? (Design: they fail together, so there is no such state.)
7. **Operations:** How will we know alerts are firing too often (the annoyance guardrail)? (Alert events table.)
8. **Compliance:** Do alerts contain amounts, and are they visible on a lock screen? (In-app only for 1.1, so not exposed; matters when push arrives.)

3. Alternatives

Alternative	Description	Verdict
X. Client-side evaluation	The app compares cached totals to limits and shows alerts	**Wrong:** client untrusted, stale, multi-device, loses alerts on retry/crash, can't be audited
Y. Server evaluates inside the save transaction (chosen)	The expense write, threshold evaluation, and alert record happen in one transaction; the response carries alerts	Meets A-1 to A-5; no new infrastructure
Z. Asynchronous evaluation	After commit, put a message on a queue; a worker evaluates and records alerts	Viable for push notifications; **rejected for 1.1:** delay, two failure modes (expense saved but job lost), needs a queue; in-app alerts need to appear immediately after the save

Driver	X	Y	Z
A-1 Atomicity	✗	✓	✗ (eventual)
A-2 Once under concurrency	✗	✓ (lock + unique)	✓ (if worker is serialised)
A-3 Retry-safe, alert not lost	✗	✓ (alerts linked to expense)	Complex
A-5 Latency within response	✓	✓ (small query)	✓ but alert arrives later
Operational burden	Low	Low	Higher (queue, worker, monitoring)

Choice: Y. Revisit when remote push is added; at that point, the alert record can act as the source for an outbox that a worker drains.

4. Data flow: save an expense that crosses 80%

```
 1. App sends POST /v1/expenses {id, amount_minor, category_id, spent_on, note}
 2. API authenticates, validates (Pydantic)
 3. BEGIN transaction
 4. INSERT expense ... ON CONFLICT (id) DO NOTHING
      - If conflict: this is a replay → jump to step 9
 5. Derive month_start from spent_on (first day of that month)
 6. SELECT limit_minor FROM budgets WHERE user_id=:u AND category_id=:c FOR
UPDATE
      - No budget: skip to step 10 (nothing to evaluate)
      - FOR UPDATE row-locks this budget so concurrent saves for the same
        category are serialised
 7. spent = SELECT COALESCE(SUM(amount_minor),0) FROM expenses
            WHERE user_id=:u AND category_id=:c
              AND spent_on >= :month_start AND spent_on < :next_month_start
              AND deleted_at IS NULL
 8. For each threshold t in (80, 100):
      if spent * 100 >= limit_minor * t      (integer arithmetic, no floats):
         INSERT INTO budget_alerts (id, user_id, category_id, month_start,
                threshold, triggered_by_expense_id) VALUES (...)
         ON CONFLICT (user_id, category_id, month_start, threshold) DO NOTHING
         RETURNING id, threshold
      Newly inserted rows = alerts fired by THIS expense.
 9. (Replay path) Read alerts WHERE triggered_by_expense_id = :id so the
    retry gets the same alerts back.
10. COMMIT
11. Respond 201 (new) or 200 (replay) with: the expense, the category's
    budget status, and alerts[] — if two thresholds fired together,
    include the highest in alerts[] to display (both are recorded).
12. App displays at most one alert per alert id and updates its cache.
```

Failure at each step:

Step	What can fail	Result
2	Bad token or invalid body	401 / 422; nothing written
3–4	DB unavailable or timeout	Error; transaction not committed; client retries same id
4	Conflict with a different user's id or different payload	Generic conflict error; no data revealed
6	Budget deleted concurrently	No row returned → no evaluation; no alert; consistent
6	Lock wait too long	`statement_timeout`/lock timeout → 503; retry
7	Slow aggregate	Bounded by indexes and timeout; max ~1,000 rows/category/month
8	Failure inserting alert	Whole transaction rolls back: no expense without its alert
10	Crash before commit	Rolled back; client retry starts clean
10–11	Commit succeeded, response lost	Retry hits conflict (step 4) → replay path returns **the same alerts**
12	App crashes before showing alert	Alert exists in DB; a status endpoint can return recent unseen alerts if needed (optional `seen_at`)

Edit and delete. Editing an expense re-runs steps 5–8 for the *new* category and month (an edit can push spending up). Deleting only lowers spending, so no alert is evaluated, and fired alerts are not withdrawn (FR-B4).

5. Data design

```sql
```

```sql
CREATE TABLE budgets (
  user_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category_id   SMALLINT NOT NULL REFERENCES categories(id),
  limit_minor   BIGINT NOT NULL
                CHECK (limit_minor > 0 AND limit_minor <= 1000000000),
  currency_code CHAR(3) NOT NULL DEFAULT 'INR',
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, category_id)          -- one limit per category per user
);

CREATE TABLE budget_alerts (
  id                      UUID PRIMARY KEY,
  user_id                 UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category_id             SMALLINT NOT NULL REFERENCES categories(id),
  month_start             DATE NOT NULL
                          CHECK (EXTRACT(DAY FROM month_start) = 1),
  threshold               SMALLINT NOT NULL CHECK (threshold IN (80, 100)),
  triggered_by_expense_id UUID REFERENCES expenses(id) ON DELETE SET NULL,
  fired_at                TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, category_id, month_start, threshold)   -- the guarantee
);

-- Starting indexes (verify with EXPLAIN; drop unused ones):
CREATE INDEX expenses_user_cat_spent_idx
  ON expenses (user_id, category_id, spent_on) WHERE deleted_at IS NULL;
CREATE INDEX budget_alerts_trigger_idx
  ON budget_alerts (triggered_by_expense_id);
```

Which constraint guarantees "at most one alert per threshold per month, even under concurrency"? The `UNIQUE (user_id, category_id, month_start, threshold)` constraint. Even if the row lock were forgotten, or two requests arrived at the same instant, the second insert would hit the unique constraint, and `ON CONFLICT DO NOTHING` would make it a no-op. **The database enforces the invariant, not the application code.** (The row lock is there for *consistent evaluation* and to avoid wasted conflicts; the unique constraint is the safety net.)

Notes: integer arithmetic (`spent * 100 >= limit * t`) avoids rounding errors. The `CHECK` uses `EXTRACT`, not `date_trunc`, because `date_trunc` on dates returns a timestamp whose behaviour depends on session settings, and constraint expressions should be deterministic. `month_start` is derived from the expense's `spent_on`, so **no time-zone arithmetic is needed on the server** for budgets, a direct benefit of ADR-002.

6. Concurrency and retries

(a) Two expenses at the same instant, two API copies. Both transactions try `SELECT ... FOR UPDATE` on the same budget row. One gets the lock; the other waits. The first computes spending, inserts the alert, commits. The second then sees committed data, computes its own spending (which now includes the first expense), and any threshold already logged is skipped via `ON CONFLICT DO NOTHING`. Result: exactly one alert per threshold, and each request's totals are consistent. *(In PostgreSQL's default READ COMMITTED isolation, the second transaction's query sees the first one's committed rows after it acquires the lock, which is why the lock matters for correct evaluation.)*

(b) Commit succeeded, response lost, client retries. The retry's insert conflicts on the expense `id` and takes the replay path. Because each alert row stores `triggered_by_expense_id`, the API returns the same alerts that the original request fired. **Without that link,** the retry would find the alert already logged, return no alert, and the user would **never see** the warning they should have received. This is the subtle trap in the exercise. The client de-duplicates display by alert `id`, so a user never sees the same alert twice.

7. Failure and security table

Item	Risk	Design response
Lock contention on a budget row	Slow saves	Lock is per (user, category); contention is only the same user's own simultaneous saves; lock timeout; monitor
Soft-deleted expenses counted	Wrong status	Every sum includes `deleted_at IS NULL`; one repository function; test
Budget removed while saving	Inconsistent state	Missing row = no evaluation; FK integrity
Old app ignores `alerts` field	No alert shown	Additive field; old apps degrade gracefully; minimum-version policy
Lowered limit below current spend	Confusing UI	Status shows "over limit"; no alert (agreed rule); open question on the user experience
Limit raised after alerts fired	Fewer alerts than user expects	Documented consequence of "once per month"; open question for Meera
Month boundary	Expense counted in wrong month	Month derived from `spent_on` DATE; boundary tests at the 1st and last day
Security: BOLA on budgets	User edits another user's budget by changing an ID	Primary key includes `user_id`; every query filters by the authenticated user; test with two users
Security: data exposure in alerts	Amounts leak on a lock screen when push is added later	In-app only now; when push is added, default to text without amounts (NFR-B3)

8. Scale triggers

Signal	Action
p95 save latency > 300 ms for 7 days, or lock waits observed	Check the execution plan of the sum query and the index; then reduce work inside the lock
`budget_alerts` growth or alert-event volume noticeably high	Review alert rules and the annoyance guardrail; consider archiving old months
Push notifications approved	Introduce an outbox + worker reading `budget_alerts`; keep the in-app path unchanged
Database CPU > 60% sustained	Scale instance up before any redesign

9. ADR-006

markdown

```
# ADR-006: Evaluate budget alerts on the server inside the expense-write transacti
**Status:** Accepted    **Date:** 2026-11-03    **Decided by:** Tech Lead

## Context
Release 1.1 adds category budgets with 80% and 100% alerts. An alert must appear
immediately after a save, must appear at most once per threshold/category/month,
must survive retries and lost responses, and must stay consistent with expenses
(NFR-B1, NFR-B5, FR-B4). Push notifications are excluded for this release.

## Options considered
A. Client-side evaluation — rejected: untrusted, stale, multi-device, loses alerts
B. Server evaluation in the same transaction as the write — chosen.
C. Asynchronous evaluation via queue and worker — rejected for 1.1: delayed alerts
   extra failure modes, extra infrastructure; reconsider with push notifications.

## Decision
In the save/edit transaction: lock the budget row, sum the month's undeleted
expenses for the category, and insert alert rows with a unique constraint on
(user_id, category_id, month_start, threshold). Alerts record the expense that
triggered them so that an idempotent replay returns the same alerts. The response
carries budget status and alerts; the client de-duplicates display by alert id.

## Consequences
+ Expense and alert change atomically; no new infrastructure.
+ The unique constraint guarantees at-most-once even if locking is removed.
- Saves do slightly more work in the transaction and hold a per-budget row lock.
- Alerts are tied to the request path; push delivery will need a separate mechanis
- "Once per month" means that raising a limit after an alert does not re-arm it
  (open question to confirm with the client).

## Revisit if
Push notifications are added; save latency at p95 exceeds target because of the
budget evaluation; budgets become shared between users.
```

What to notice in the reference. The design introduced **no new services, no queue, no cache.** It solved a hard problem (atomic, at-most-once, retry-safe alerts) using a transaction, a row lock, a unique constraint, and one extra column. It also *surfaced two product questions* (limit raised after an alert; lowered below spend) that go back to the client. That is the cycle: requirements shape design, and design exposes new requirements.

Part 9 — Recall test, with answers

Cover the answers and try first.

Q1. What is an architecturally significant requirement? Give one from Spendwise and one that is *not* architecturally significant. *Answer:* A requirement that would force a different structure if it changed. "Exactly one expense on retry" is significant (affects API, IDs, constraints). "A note is at most 200 characters" is not.

Q2. Why did the back-of-envelope estimate matter? *Answer:* It showed that load is tiny (a few writes per second, about a gigabyte per year), so the real design drivers are correctness, security, operability, and mobile release constraints, not scale. It prevents building Kubernetes and caches for a problem that does not exist.

Q3. Give two reasons for choosing a modular monolith over microservices here, and one condition that would change the decision. *Answer:* One database and transaction make correctness (no duplicates, consistent totals) simple; a solo engineer cannot sensibly operate many services. Revisit if multiple teams need independent releases, if one feature needs very different scaling, or if offline becomes a Must.

Q4. Why must the client never talk to the database directly, and why is the API considered the only trusted place for rules? *Answer:* The client is untrusted and can be bypassed by anyone; direct access cannot be revoked once apps are installed. Authentication, authorisation, validation, and business rules must be enforced where users cannot tamper with them.

Q5. Explain how a client-generated UUID, a primary key, and `ON CONFLICT` together make a retry safe. *Answer:* The retry carries the same ID. The primary key rejects a second row. The API reads back the existing row (if same user and content) and returns it, so the user sees exactly one expense even when the first response was lost.

Q6. Why store `spent_on` as a DATE rather than a UTC timestamp? *Answer:* The user's intent is a calendar date. A UTC timestamp makes month grouping depend on time-zone arithmetic and moves expenses between months for travellers. A DATE gives a simple, index-friendly range query. The cost is losing time of day, which no requirement needs.

Q7. Why compute monthly totals on read instead of keeping a totals table? *Answer:* A separate table creates a second source of truth that can disagree with the expenses, violating NFR-4. The aggregate is cheap at this scale. Add pre-aggregation later if measurements demand it.

Q8. What is BOLA, and what structural defence do we use? *Answer:* Broken object-level authorisation: the API returns or changes a record by ID without checking ownership. Defence:

every repository query filters by the authenticated user's ID; optionally Row-Level Security as defence in depth; tests with two users.

Q9. Why do ADRs include "alternatives" and "revisit if"? Why are ADRs not edited later? *Answer:* Alternatives make the decision credible and challengeable; "revisit if" says what evidence would change it. ADRs are historical records; if a decision changes you write a new ADR that supersedes the old one, preserving why you once thought otherwise.

Q10 (challenge). A client asks for 99.99% availability and says "just use microservices and Kubernetes, that's what big companies do." How do you respond? *Answer:* Explain that 99.99% allows about four minutes of downtime per month and requires redundancy at every layer, with real cost and operational effort that ten pilot users do not justify. Microservices and Kubernetes solve organisational and independent-scaling problems we don't have and add failure modes. Offer the pilot target (99.5%) with a written, costed upgrade path and a revisit trigger, such as paying customers or a user threshold, and record the decision in an ADR.

Part 10 — Completion checklist

Tick honestly. Revisit the section noted for any you can't.

- ☐ I can define **architecture** as "decisions expensive to change" and separate it from design and implementation (3.1).
- ☐ I can extract **ASRs** from a PRD and explain why some NFRs are not architecturally significant (3.1, 4.2).
- ☐ I can do a **back-of-envelope load estimate** and say what it implies (3.2).
- ☐ I can list **real alternatives**, compare them against drivers, and justify a choice (3.3).
- ☐ I can state the **boundary rules** (client untrusted, only the API touches the DB, stateless API, one owner per table, layering) and why each exists (3.4).
- ☐ I can trace the **save-expense flow** and explain idempotency, transactions, and constraints (3.5).
- ☐ I can build a **failure table**: what fails, what the user sees, what happens to the data, how we find out (3.6).
- ☐ I can run a **light threat model** and explain **BOLA** and its structural defence (3.7).
- ☐ I can distinguish **cheap insurance** from **speculative construction** and define **scale triggers** (3.8).
- ☐ I can reason about **operations**: managed vs self-hosted DB, backups and restore drills, environments, migrations, observability, mobile release constraints (3.9).
- ☐ I can write a right-sized **TDD** and **ADR** with traceability to requirement IDs (3.10, 4.3, 4.4).
- ☐ I can explain why architecture choices are **context-dependent** and name the conditions that would change mine (3.3, 6).

Closing: what you should understand before moving on

Before the next lecture, make sure you can say these five things in your own words:

1. **Architecture is the set of expensive-to-change decisions.** Start from the few requirements that drive structure, not from a list of technologies.
2. **Estimate before you design.** A ten-minute calculation can show that your real risks are correctness, security, and operability, not scale.
3. **Boundaries are rules, not boxes.** The client is untrusted; the API owns the rules; the database enforces the invariants; one table has one owner.
4. **Design every important flow around failure.** For each step ask: what fails, what does the user see, what happens to the data, how do we know?
5. **Write down the why.** The TDD says what the design is; ADRs say why, with real alternatives, honest negatives, and conditions to revisit.

This is the end of Module 3, Lecture 1. I'm stopping here, as instructed.